Speculative GHR Forwarding: Eliminating Stale Branch-Predictor State in Deep FPGA Pipelines

Devansh Joshi and Shafin Ula Department of Electrical and Computer Engineering,
The University of Texas at Austin
Email: {devansh.jos, shafin.ula}@utexas.edu

***Abstract*—Deeper pipelines raise an FPGA soft core's clock frequency, but they make every branch misprediction more costly. As depth grows, the global history register (GHR) of a correlation-based branch predictor takes longer to update. The register grows stale and inflates mispredictions, though the predictor hardware is unchanged. We eliminate this penalty with Speculative GHR Forwarding (SGF). Its key idea is that an in-order pipeline's existing stage registers already hold the per-branch state required for rollback. Recovery therefore needs no reorder buffer, unlike the out-of-order schemes that inspired it. We evaluate SGF on five RISC-V pipelines of four to eight stages, from identical functional-unit hardware on a Xilinx Artix-7 device. The experiment separates this cost into an inherent flush penalty and a correctable stale-GHR penalty. A history-free control predictor and a thirty-two-fold capacity sweep confirm the latter causally. On the seven-stage pipeline, SGF reduces mispredictions by 31 percent at under 1 percent area on single-history predictors, with no frequency penalty; on multi-table predictors like TAGE it is higher. The cycles-per-instruction gain is modest: SGF is a favorable cost-benefit tradeoff, not a large speedup. The same experiment finds the execute-stage split to be the only frequency knee, making a six-stage pipeline throughput-optimal.**

***Index Terms*—RISC-V, pipeline depth, FPGA, Artix-7, RV32IM, branch prediction, gshare, speculative GHR forwarding, microarchitecture**

I. Introduction

How deep should an FPGA pipeline be? On ASIC processes the answer is well studied: partition the datapath until gate delay per stage reaches 6–8 FO4 inverter delays, then stop because branch-misprediction penalties and power begin to dominate [1]–[3]. On FPGAs, where LUT delay is largely function-independent and routing delay dominates timing [4], [5], that answer does not transfer directly. This paper answers the question with a controlled experiment on Artix-7 and reports three findings: pipeline depth raises clock frequency only through the execute-stage split; beyond it, deeper pipelines add a stale global-history penalty on branch-dense workloads; and a lightweight fix, Speculative GHR Forwarding (SGF), removes that penalty at under 1% area and no frequency loss. Prior RISC-V soft cores differ in ISA support, predictor design, memory architecture, and forwarding strategy, making performance differences across projects unattributable to any single factor. We are not aware of a published study that varies pipeline depth as the single controlled design parameter on a common FPGA target while holding the RTL of all other microarchitectural modules constant. Each stage split does carry structural side effects (an added load-use stall, a fetch bubble, an extra forwarding source), which we make explicit and decompose rather than assume away.

A subtler cost compounds the explicit penalty of deeper pipelines: the predictor's global history register (GHR) becomes stale because the latency between branch resolution and the next prediction spans more cycles. Speculative history updating is itself an established technique. Yeh and Patt's two-level adaptive predictor [6] already updates history speculatively, and ASIC out-of-order processors (Alpha 21264 [7], MIPS R10000 [8]) recover it on misprediction through the reorder buffer (ROB) or a dedicated branch stack. Gonzalez and Horowitz quantified the staleness cost in ASIC simulations [9]. These recovery mechanisms are tied to the out-of-order substrate, however: the R10000's branch stack checkpoints the register-map alias table and the Alpha's recovery rides the ROB. These are centralized structures that an in-order FPGA soft core lacks and could not add cheaply (the structures alone would dwarf the predictor they protect). To our knowledge, the effect itself has not been measured under controlled conditions on FPGA fabric, and no lightweight in-order realization of speculative history has been demonstrated there.

This paper makes two contributions. First, we construct a controlled experiment that cleanly separates the CPI cost of deeper pipelining into two mechanisms. Five RV32IM pipeline variants (4–8 stages) share identical RTL for all functional units, synthesized on Xilinx Artix-7 XC7A35T across 50 post-place-and-route runs with seven benchmark workloads:

- **Mechanism A (inherent):** Higher per-misprediction flush penalty from additional stages to drain. Present on all workloads, predictable from pipeline structure alone.
- **Mechanism B (correctable):** Inflated misprediction count from stale GHR state. Present only on branch-dense, data-dependent workloads, two of our seven (+5.7% on CoreMark at 6-stage, +10% on statemate at 7/8-stage), and unpredictable without cycle-accurate simulation.

Two controls establish that Mechanism B is causal rather than correlational: a bimodal predictor (no GHR) shows *zero* depth-dependent inflation on the same workloads, and a 32× PHT capacity sweep shows the inflation is not a table-size aliasing artifact (Section VII-A). A first-principles CPI model ($R^2 = 0.977$, $R^2_{\mathrm{CV}} = 0.941$) captures the depth decomposition (Section IV-E).

Second, we propose Speculative GHR Forwarding (SGF), an implementation and measurement contribution rather than a new predictor. Speculative history with checkpoint restore is

established [6], [7]; SGF does not reinvent it. The novelty is twofold. The first part is the *in-order, ROB-free realization*. An in-order single-issue pipeline admits no nested or out-of-order rollback (Section V-C), so a single GHR checkpoint forwarded through the *existing* pipeline registers is provably sufficient. It replaces the reorder buffer or branch stack that ASIC schemes require. The second is the *controlled demonstration* that stale GHR state measurably degrades deep pipelines on Artix-7, isolated from the inherent flush penalty so that SGF removes only the correctable Mechanism B. On the 7-stage pipeline SGF cuts mispredictions 31.4% on CoreMark and 22.7% on statemate, pushing deep-pipeline counts *below* the shallow-pipeline baseline, at 49 LUTs and 57 FFs (0.7% area, no frequency loss). The benefit reproduces on a tournament and a downscaled TAGE predictor (Section V-K); the sub-1% cost holds for single-history predictors and grows on multi-table TAGE.

The value is a favorable cost-benefit tradeoff, not a large speedup. Mechanism B is only one of several depth costs, so the 31% misprediction cut is a roughly 3% CPI gain. SGF also does not make a deeper pipeline out-throughput a shallower one: the 6-stage stays the highest-throughput depth even against a 7-stage with SGF. Its benefit appears when depth is *already forced* by a timing constraint, such as a registered-BRAM instruction fetch that mandates the IF1/IF2 split. There the stale-GHR penalty is an unavoidable cost of a decision made for frequency. SGF removes it for under 1% area and no frequency loss, so it is a low-risk addition to any gshare-class in-order pipeline with three or more stages of update latency. The gain is largest with tightly coupled instruction and data memory, the common embedded soft-core case; cache-miss stalls dilute it, and the area cost rises on multi-table predictors (Sections VI-C, V-K).

The experiment also reveals a two-tier frequency structure on Artix-7. Shallow pipelines (4/5-stage) cluster at 70–74 MHz; deep pipelines (6/7/8-stage) cluster at 115–118 MHz ($p < 0.001$, Cohen's $d = 18.5$). The execute-stage split at depth 6 is the sole significant frequency knee, which makes the 6-stage the throughput optimum, a directly actionable design rule for any RV32IM-class FPGA soft core independent of SGF.

II. Background and Related Work

A. RISC-V ISA and Pipeline Depth Tradeoffs

RV32IM provides a fixed-width 32-bit encoding with regular operand fields well suited for pipelined implementation [10], [11]. Deeper pipelines reduce per-stage logic delay but increase misprediction penalty, area, and forwarding complexity [1], [12]. Hrishikesh et al. established 6–8 FO4 delays per stage as optimal for ASICs [1]; Sprangle and Carmean showed diminishing returns beyond certain depths [2]. Both assume ASIC timing where gate delay scales linearly with logic depth. A less studied effect is that deeper pipelines also degrade branch prediction accuracy: the increased latency from resolution to the next prediction leaves the GHR stale when branches are closely spaced. Gonzalez and Horowitz identified this in ASIC simulations [9]; we are not aware of a study measuring it on FPGA fabric under controlled conditions.

TABLE I
Representative RISC-V FPGA soft cores. None varies pipeline depth as a controlled parameter with invariant functional-unit hardware, and none addresses stale-GHR recovery.

Core	Pipeline	Predictor	Depth-config.?
SERV [13]	bit-serial	none	no
PicoRV32 [14]	multi-cycle	none	no
Ibex [15]	2-stage	bimodal-style	no
VexRiscv [16]	configurable	optional	not indep.
SweRV EH1 [17]	2-wide superscalar	BHT	no
CVA6 [18]	out-of-order	gshare-class	no
This work	4–8 (shared RTL)	gshare+SGF	yes

B. FPGA Architecture Considerations

On Xilinx 7-series FPGAs, LUT delay is largely function-independent, and routing delay often exceeds LUT delay [4], [5]. Splitting a combinational path across two pipeline stages therefore yields frequency improvement only when the split targets LUT-dominated logic; routing-dominated paths see diminishing returns [5]. DSP48E1 blocks provide registered multiply-accumulate; Block RAM offers single-cycle synchronous reads.

C. Related RISC-V Soft Cores

Prior RISC-V FPGA implementations vary widely in scope (Table I). They span bit-serial and multi-cycle designs with no pipeline, fixed shallow pipelines, and out-of-order or superscalar cores. None lets pipeline depth vary as a single controlled parameter with the predictor, MDU, and memory hardware held invariant, and none addresses stale-GHR recovery. That combination is what this study requires and supplies.

D. Branch Predictors and Speculative History

Correlation predictors key a pattern-history table on global branch history. gshare [19] XORs that history with the PC; tournament predictors add a chooser between global and local components; TAGE [20] uses several tagged tables at geometric history lengths; and perceptron predictors [21] are a neural alternative. All of them index on a global history register, so all are exposed to the stale-GHR effect this paper isolates, and we measure three of the four.

Speculative GHR updating with checkpoint rollback is long established and is *not* what we claim as new. Yeh and Patt [6] describe speculative history in their two-level predictor. Out-of-order ASICs recover it through the reorder buffer or a branch stack (Alpha 21264 [7], MIPS R10000 [8]), and in-order ASIC pipelines have long used a shift-register history with checkpoint/mask restore on misprediction. SGF's contribution is therefore the FPGA realization, not the rollback concept. The ASIC recovery structures either tie to out-of-order state (the R10000's branch stack checkpoints the register-rename map, the Alpha rides ROB squashing) or assume a custom ASIC datapath. SGF instead reuses the in-order soft core's *existing* pipeline registers as the checkpoint-forwarding path, adding no dedicated recovery hardware. We also measure

the effect it corrects under controlled conditions. In tagged predictors stale GHR corrupts both index and tag (a different error profile than gshare), needing per-table checkpoints [20], [22]; Michaud et al. [23] note that update latency compounds with PHT aliasing.

E. FPGA Pipeline Studies

Prior surveys have cataloged pipelining techniques across RISC families [24], and configurable generators such as Rocket Chip [25] offer parameterizable pipelines with branch-prediction front-ends. Such generators and other in-order soft cores do carry lightweight branch-recovery state (PC redirect and BTB/RAS checkpoints). But none, to our knowledge, speculatively checkpoints and forwards the *global history register* of a correlation predictor in an in-order FPGA pipeline. The out-of-order designs that recover speculative history do so through the ROB or a branch stack, the centralized structures an in-order soft core lacks. Neither line of work isolates pipeline depth as a controlled FPGA variable, nor addresses the stale-GHR effect. The FPGA-specific study of Kuon and Rose [5] characterizes the LUT-routing delay gap that makes FPGA pipelining fundamentally different from ASIC pipelining, but does not examine branch-prediction interactions.

To our knowledge the gap is twofold. No published controlled experiment varies pipeline depth as the single controlled design parameter on a single FPGA device, holding functional-unit RTL constant and decomposing each split's side effects rather than claiming them away. And no lightweight mechanism corrects the stale-GHR degradation such an experiment reveals.

III. Experimental Setup

We keep three result types distinct throughout. $F_{\max}$, area, and power are *post-place-and-route* (static timing analysis and Vivado vector-based power estimates). Cycle and misprediction counts are from *RTL simulation*. The CPI decomposition (Section IV-E) is *analytical*, and where it takes a measured misprediction rate as input we say so. Tables and text label which category each number belongs to.

A. Common Microarchitecture

All five pipeline variants share the following modules, instantiated from identical RTL source files:

ALU. A combinational 10-operation arithmetic logic unit supporting ADD, SUB, AND, OR, XOR, SLT, SLTU, SLL, SRL, and SRA.

Multiply-Divide Unit (MDU). A 2-cycle pipelined multiplier with registered input and output stages, designed to infer DSP48E1 blocks. The divider uses a 32-cycle restoring algorithm. Supported operations are MUL, MULH, MULHSU, MULHU, DIV, DIVU, REM, and REMU. Division by zero and signed overflow follow the RISC-V specification [10].

Branch Predictor. A gshare predictor [19] with a 64-entry pattern history table (PHT) of 2-bit saturating counters [26], a 6-bit global history register (GHR) in the two-level adaptive style [6], a 32-entry direct-mapped branch target buffer (BTB), and a 4-entry return address stack (RAS). The PHT is indexed by XORing the lower PC bits with the GHR. The predictor hardware is identical in all five variants. However, the number of pipeline stages between the fetch stage (where predictions are made) and the execute stage (where predictions are resolved and the PHT is updated) differs across variants. This update latency is the central variable in our analysis and the target of SGF.

Predictor scale justification. The 64-entry PHT and 6-bit GHR are deliberately sized for the resource-constrained embedded soft-core context that is the target application of this work. This configuration consumes 128 LUT bits (64 $\times$ 2-bit counters) and occupies $<$1% of the Artix-7 fabric. Larger predictors (e.g., 1024–4096-entry PHTs with 10–12-bit GHRs) are standard in high-performance ASIC processors but would dominate the area budget of a $\sim$7,500-LUT soft core. Two controls in Section VII-A establish that the depth effect we attribute to this predictor is caused by update latency rather than its small size: a 32$\times$ PHT sweep (Section VII-A1) and a no-GHR bimodal predictor (Section VII-A2). A 6-bit history is short relative to high-performance gshare but representative of low-end embedded cores, which use minimal or no global history (e.g., Ibex's bimodal-style predictor, SweRV's modest branch-history table). It also bounds the speculative-forwarding checkpoint to 6 bits per in-flight branch.

CSR Unit. Machine-mode control and status registers including `mstatus`, `mtvec`, `mepc`, `mcause`, `mscratch`, and 64-bit hardware performance counters for cycles (`mcycle`), retired instructions (`minstret`), total branches, and branch mispredictions. Each benchmark reads these counters to obtain cycle counts, instruction counts, branch counts, and misprediction counts.

Instruction Memory. A 4096-word (16 KB) ROM initialized at elaboration via `$readmemh`, sized to hold the largest benchmark. A 64-line direct-mapped instruction cache with combinational hit logic sits between the PC and the instruction memory.

Data Memory. A 2048-word (8 KB) synchronous RAM supporting byte, halfword, and word loads (signed and unsigned) and stores.

Register File. A 32$\times$32-bit register file with two read ports and one write port, including a write-through bypass for simultaneous read and write to the same register.

B. Pipeline Variants

The five variants are constructed by splitting or merging stages of a common datapath. Each split targets a specific combinational path:

4-stage (IF, ID, EX, MEM/WB): Merges memory access and write-back. Load data returns at the end of the combined MEM/WB stage, in time to forward to the next instruction's EX, which eliminates the load-use *stall* but creates the longest critical path. One forwarding source (MEM/WB$\rightarrow$EX). Branch penalty: 2 cycles.

5-stage (IF, ID, EX, MEM, WB): Classic RISC organization [27]. Introduces a 1-cycle load-use stall. Two forwarding sources. Branch penalty: 2 cycles.

6-stage (IF, ID, EX1, EX2, MEM, WB): Splits the execute phase, separating the forwarding multiplexer (EX1) from ALU and branch comparator (EX2). Three forwarding sources. Branch penalty: 3 cycles. This split targets the dominant critical path in the 4/5-stage designs.

7-stage (IF1, IF2, ID, EX1, EX2, MEM, WB): Splits instruction fetch, placing branch prediction in IF2 with a registered PC. Introduces a 1-cycle IF2 bubble on every correctly predicted taken branch (IF1's speculative fetch is discarded). Branch penalty: 4 cycles. This split targets BRAM read latency. An industrial pipeline might deploy a zero-delay Next-Line Predictor (NLP) or BTB directly in IF1 to eliminate this bubble. We do not, because embedding specialized routing paths in IF1 would alter the physical timing paths unevenly across the 4-to-8-stage variants. The bubble is a deliberate control-variable constraint that preserves strict structural invariants across all depths. It is also orthogonal to SGF: an IF1 next-line predictor changes only *where* the next fetch address is produced, not *when* the GHR is updated at resolution. It would remove the fetch bubble (the $\beta\, B(D)\, f_b$ term) but leave the stale-GHR window, and hence SGF's misprediction reduction, unchanged. Its effect would be to lower the baseline CPI that SGF is measured against, which shrinks SGF's *relative* CPI gain slightly without changing its misprediction counts.

8-stage (IF1, IF2, ID, EX1, EX2, MEM1, MEM2, WB): Splits memory access, creating a dual load-use stall (loads not available until MEM2). Branch penalty: 4 cycles. The MEM split lengthens the load-use path but leaves the fetch-to-resolution path unchanged, so the predictor's update latency (4 stages), and therefore Mechanism B, is identical to the 7-stage; the 8-stage's only added cost over the 7-stage is the dual load-use stall, not more stale history. This is why the 7- and 8-stage baselines share misprediction counts on every workload.

Table II summarizes the per-event costs. The 7/8-stage variants share the same branch penalty because both resolve branches in EX2 with equal numbers of preceding stages. The predictor update latency, the number of stages between prediction and resolution, is the critical parameter for Mechanism B.

Threats to isolation. Each split introduces structural side effects (load-use stall changes, IF2 bubbles, forwarding-source count) inseparable from the depth increase. Table II makes these explicit; the CPI model (Section IV-E) decomposes each effect into a separate term. The claim is not that depth alone causes CPI increase, but that each split's Mechanism B component is correctable.

C. Synthesis Methodology

To quantify placement-dependent variance in $F_{\max}$, each variant is synthesized and implemented 10 times with different placement and routing directive combinations (Default, Explore, ExtraNetDelay_high/low, ExtraPostPlacementOpt, NoTimingRelaxation, and AggressiveExplore in various pairings). This produces 50 independent implementation runs (5 variants $\times$ 10 directive combinations). The target device is Xilinx Artix-7 XC7A35TCPG236-1. All syntheses use Vivado 2025.2 in out-of-context (OOC) mode with a 5 ns clock constraint (200 MHz target) to drive the tools toward maximum optimization effort. The reported $F_{\max}$ values are means with standard deviations and 95% confidence intervals computed via Student's t-distribution with 9 degrees of freedom. $F_{\max}$ for each run is derived from the worst negative slack (WNS) as $F_{\max} = 1000/(5 - \text{WNS})$.

Seed selection. The 10 placement/routing directive pairings were fixed *before* any synthesis run was executed and were not modified or re-selected after observing results. The pairings were chosen to sample the space of Vivado optimization heuristics, not to optimize for any particular variant. We treat these pairings as a fixed, pre-registered sample. Run-to-run dispersion (mean, standard deviation, t-based 95% CI) is reported as sensitivity to *these seven directives*. It is not an estimate over the population of all achievable placements or a random sample of Vivado's optimization space. This distinction matters for absolute spreads, but not for cross-variant comparisons. Because every variant (baseline and SGF) is synthesized under the *same* directive set, an SGF-vs-baseline $F_{\max}$ comparison is paired and controlled regardless of how representative the set is. Ten runs is a modest sample: it is sufficient to separate the 44 MHz inter-tier gap (Cohen's $d = 18.5$, non-overlapping CIs) from within-tier noise, but not to resolve the sub-2 MHz intra-tier differences, which we accordingly do not interpret.

Simulation environment. All cycle counts and misprediction counts are from Vivado XSim behavioral (RTL) simulation. The design is a single-clock-domain synchronous pipeline with no asynchronous interfaces, so behavioral simulation is *expected* to reproduce the synthesized design's cycle count. We present this as an assertion grounded in the deterministic single-clock structure, not a hardware-validated result (no physical-board cycle measurement is reported here; Section VIII). We scope it deliberately. Simulation validates *cycle counts*, and hence CPI, but not physical timing; physical timing is established separately by post-place-and-route static timing analysis (the source of all $F_{\max}$ values). We never treat a cycle-accurate simulation as evidence of timing closure. Determinism is verified by identical instruction counts, branch counts, and memory checksums across all five depth variants for every workload.

Reproducibility. All benchmarks are compiled with `riscv-none-elf-gcc` 15.2.0 (xPack) using `-march=rv32im_zicsr -mabi=ilp32 -O1 -nostdlib -ffreestanding`. CoreMark uses the official EEMBC source (`github.com/eembc/coremark`) with a bare-metal porting layer; Embench-IoT uses the official source [28]. Linker scripts place code at address 0 (IMEM) and data at 0x10000 (DMEM); the startup routine clears BSS and initializes seed variables before calling `main`. The 10 placement/routing directive pairings are: (1,5) Default/Default ($\times$2), (2) Explore/Explore, (3) Default/NoTimingRelaxation, (4) ExtraNetDelay_high/Explore, (6) ExtraPostPlacementOpt/AggressiveExplore, (7,10) Explore/NoTimingRelaxation ($\times$2), (8) ExtraNetDelay_low/Explore, (9) Default/AggressiveExplore. Duplicate pairings (1/5 and 7/10) use different Vivado random seeds, producing

TABLE II
PIPELINE STRUCTURE AND PER-EVENT HAZARD COSTS ACROSS DEPTHS (ARCHITECTURAL PARAMETERS, NOT MEASURED DATA). PREDICTOR UPDATE LATENCY (BOTTOM ROW) IS THE CENTRAL VARIABLE FOR MECHANISM B.

Property	**4-stg**	**5-stg**	**6-stg**	**7-stg**	**8-stg**
Pipeline registers	3	4	5	6	7
Forwarding sources	1	2	3	3	3
Load-use stall	0	1 cy	1 cy	1 cy	1–2 cy
Branch penalty	2 cy	2 cy	3 cy	4 cy	4 cy
IF2 prediction bubble	0	0	0	1 cy	1 cy
Predictor update latency	2	2	3	4	4

distinct placements. Complete RTL, synthesis TCL scripts, benchmark hex files, and result logs are archived at `github.com/devanshjoshi08/RISCV-Pipeline-Research`.

D. Benchmark Suite

Seven workloads from three standard sources measure CPI and branch prediction behavior across all five depths. All are compiled at `-O1`; higher optimization levels may alter branch density and correlation structure (see Section VIII). Each benchmark reads the hardware performance counters (`mcycle`, `minstret`, branch count, misprediction count) before and after the workload, computes deltas, and stores results to data memory for testbench readout. Functional correctness is verified by confirming that all five pipeline variants produce identical instruction counts, branch counts, and memory checksums for each workload.

The seven workloads, listed as (instructions, branches, branch frequency), are: **Dhrystone 2.1** [12] (2,926, 400, 13.7%); a mixed **Diagnostic** program exercising arithmetic, load/store, calls, and conditional branches (1,367, 360, 26.3%); official EEMBC **CoreMark** [29] (10 iterations, PERFORMANCE_RUN; 2,893,145, 719,810, 24.9%); and four Embench-IoT [28] kernels, **aha-mont64** (Montgomery multiply, multiply-heavy; 4,453,858, 512,593, 11.5%), **crc32** (4,008,648, 174,591, 4.4%; near-perfectly predictable, 0.19% misprediction rate), **statemate** (FSM with data-dependent transitions; 3,160,225, 376,291, 11.9%), and **edn** (signal processing; 2,972,527, 338,986, 11.4%).

IV. BASELINE RESULTS

This section establishes the baseline measurements that motivate SGF. The synthesis results reveal the FPGA-specific frequency structure; the CPI measurements expose Mechanism B as the target for correction.

A. Synthesis Results

Table III presents the post-place-and-route results for all five variants, with $F_{\max}$ means, standard deviations, and 95% confidence intervals.

The results reveal a two-tier frequency structure: the shallow tier (4/5-stage) at 70–74 MHz and the deep tier (6/7/8-stage) at 115–118 MHz. The 44 MHz gap ($p < 0.001$, Cohen's $d = 18.5$) confirms this is not a placement artifact. The effect size is extreme by convention ($d > 0.8$ is "large"), but expected here: the two tiers' 95% confidence intervals do not overlap, so a large standardized separation is the arithmetic consequence of the small within-tier placement variance.

The jump at depth 6 results from splitting the execute stage: the 4/5-stage critical path traverses the forwarding mux, ALU, and branch comparator in series. The EX1/EX2 split halves this combinational depth, yielding 54–56% frequency improvement. Further splits yield no additional $F_{\max}$ because routing delay, not reduced by register insertion [5], dominates the remaining paths. Within the deep tier, differences are $<$2 MHz ($<$2%) with limited practical impact.

B. Area

The 4-stage uses the fewest resources (7,219 LUTs, 8,040 FFs; Table III). Each additional pipeline stage adds 52–210 LUTs and 76–150 FFs, for a total increase from 4 to 8 stages of 622 LUTs (8.6%) and 638 FFs (7.9%). All five variants use 12 DSP48E1 blocks for the pipelined multiplier, and no BRAM tiles since both instruction and data memories are implemented in distributed LUT RAM.

C. CPI Measurements

Table IV reports CPI for all seven workloads across the five depths; it rises monotonically with depth on every workload, and the analytical model of Section IV-E reproduces these values. Dhrystone and Diagnostic produce *identical* misprediction counts at all depths (203/400 and 69/360 branches respectively): their branches are spaced widely enough that update latency adds none, so only Mechanism A operates. The 5-stage CPI exceeds the 4-stage despite an identical branch penalty because of its 1-cycle load-use stall (absent in the 4-stage merged MEM/WB).

CoreMark, by contrast, exhibits a depth-dependent rise in misprediction count, the signature of Mechanism B. The 6-stage produces 154,077 mispredictions (21.4%) versus 145,735 (20.2%) at 4/5-stage, a 5.7% increase from the longer update latency. The 7- and 8-stage are *identical* at 147,760 (20.5%): the two deepest pipelines share the same 4-stage predictor update latency, so the 8-stage's extra MEM split adds flush-penalty cycles (Mechanism A) but no new staleness (Mechanism B). All five variants execute identical instruction (2,893,145) and branch (719,810) counts, and the official CoreMark/MHz score ranges from 2.23 (4-stage) to 1.58 (8-stage). This inflation is the specific target of SGF.

Among the four Embench-IoT kernels, multiply-heavy aha-mont64 has the lowest CPI (few branches, MDU-bound). crc32 has almost zero mispredictions (344 across all depths,

TABLE III
POST-PLACE-AND-ROUTE SYNTHESIS RESULTS ON ARTIX-7 XC7A35T (MEAN ± STD FROM 10 SYNTHESIS DIRECTIVE COMBINATIONS, 50 RUNS TOTAL). 95% CONFIDENCE INTERVALS COMPUTED VIA STUDENT'S t WITH 9 D.F. TIER GAP: TWO-SAMPLE t-TEST $p < 0.001$, COHEN'S $d = 18.5$.

Metric	4-stg	5-stg	6-stg	7-stg	8-stg
$F_{\max}$ (MHz)	70.4	74.0	117.3	115.0	117.5
± std (MHz)	2.4	1.5	1.4	2.0	1.8
95% CI (MHz)	[68.7, 72.1]	[72.9, 75.1]	[116.3, 118.3]	[113.5, 116.5]	[116.2, 118.8]
Slice LUTs	7,219	7,429	7,631	7,766	7,841
Slice Regs (FFs)	8,040	8,190	8,501	8,550	8,678
DSP48E1	12	12	12	12	12

TABLE IV
CPI FROM RTL SIMULATION ACROSS ALL SEVEN WORKLOADS AND FIVE PIPELINE DEPTHS (f_b = BRANCH FREQUENCY). EVERY VARIANT EXECUTES IDENTICAL INSTRUCTION, BRANCH, AND MISPREDICTION COUNTS PER WORKLOAD (DETERMINISM CHECK). COREMARK AND STATEMATE ALSO SHOW DEPTH-DEPENDENT MISPREDICTION INFLATION (MECHANISM B; SEE TEXT AND FIG. 1).

Workload (f_b)	4-stg	5-stg	6-stg	7-stg	8-stg
Dhrystone (13.7%)	1.41	1.48	1.68	1.85	1.99
Diagnostic (26.3%)	1.38	1.60	1.66	1.89	2.11
CoreMark (24.9%)	1.54	1.63	1.83	2.02	2.18
aha-mont64 (11.5%)	1.21	1.21	1.25	1.32	1.32
crc32 (4.4%)	1.30	1.30	1.39	1.52	1.56
statemate (11.9%)	1.21	1.30	1.37	1.50	1.69
edn (11.4%)	1.48	1.51	1.66	1.78	1.95

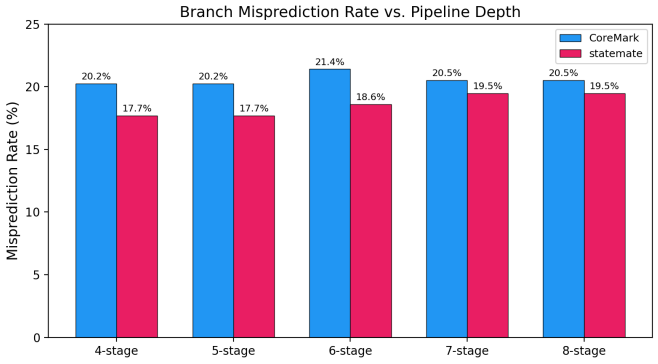

Fig. 1. Branch misprediction rate (%, RTL simulation) on CoreMark and statemate across five depths. The rate climbs with depth, CoreMark at the 6-stage (20.2%→21.4%), statemate at the 7/8-stage (17.7%→19.5%), which is the Mechanism B inflation. The five other workloads (not shown) hold a constant rate at all depths (Mechanism A only).

0.19%) on its highly predictable inner loop. statemate shows the strongest depth-dependent effect (66,603 mispredictions at 4/5-stage vs. 73,263 at 7/8-stage, +10.0%) from data-dependent state transitions. edn has moderate CPI with a low 3.1% misprediction rate.

D. Branch Misprediction Analysis: Mechanism A vs. Mechanism B

Deeper pipelines affect CPI through two distinct mechanisms that must be analyzed separately. Their separation is the measurement contribution of this paper and the foundation for SGF.

Mechanism A: Higher per-misprediction penalty. Each misprediction flushes all stages between fetch and resolution. The 4/5-stage designs flush 2 stages (2 wasted cycles), the 6-stage flushes 3, and the 7/8-stage flush 4. This cost scales linearly with the number of flushed stages and is independent of predictor accuracy. On Dhrystone (203 mispredictions, 2,926 instructions), the per-misprediction CPI contribution increases from $203 \times 2/2926 = 0.14$ at 4-stage to $203 \times 4/2926 = 0.28$ at 7/8-stage.

Mechanism B: Inflated misprediction count from stale GHR state. In deeper pipelines, the latency between branch resolution (where the PHT is updated) and the next prediction (where the PHT is read) spans more cycles. If two branches are closely spaced, the second branch may be predicted using a GHR that has not yet incorporated the first branch's outcome. This produces *additional* mispredictions that would not occur in a shallower pipeline with the same predictor hardware.

Fig. 1 separates these effects. On five of seven workloads (Dhrystone, Diagnostic, aha-mont64, crc32, edn), the misprediction count is identical across all five depths, only Mechanism A operates. On CoreMark, the 6-stage produces 154,077 mispredictions versus 145,735 for the 4/5-stage (5.7% more), adding Mechanism B on top of Mechanism A. On statemate, the 7/8-stage pipelines produce 73,263 versus 66,603 for the 4/5-stage (10.0% more).

Mechanism B requires two conditions: (1) branches spaced closely enough that a new prediction occurs before the prior branch's resolution updates the PHT, and (2) outcomes sensitive to global history. A useful proxy metric is the *average inter-branch distance* (IBD), defined as IBD $= N/B$ where N is total instructions and B is total branches. Table V classifies our workloads by IBD and observed Mechanism B behavior. The two workloads exhibiting Mechanism B (CoreMark, statemate) have IBD ≤ 8.4 and data-dependent branch outcomes. Workloads with IBD > 8.4 or highly biased outcomes (crc32: 99.8% taken) show no depth-dependent misprediction variation regardless of branch density. This suggests a *preliminary* indicator, not a validated design rule: SGF is likely beneficial when IBD < 10 *and* branches are data-dependent rather than loop-structured. Because this rests on only the two Mechanism-B workloads in our suite and is not validated on held-out workloads, we present it as an observation to test rather than a guideline to apply.

SGF eliminates precisely this unpredictable amplification on susceptible workloads. Two controls in Section VII-A confirm the cause is GHR staleness, not an artifact: a no-GHR bimodal predictor shows *zero* depth-dependent inflation on the same

TABLE V
MECHANISM B SUSCEPTIBILITY CLASSIFIED BY INTER-BRANCH DISTANCE (IBD) AND BRANCH OUTCOME PREDICTABILITY.

Workload	IBD	Outcome type	Mech. B?
CoreMark	4.0	Data-dependent	Yes (+5.7%)
statemate	8.4	Data-dependent	Yes (+10%)
edn	8.8	Mixed	No
aha-mont64	8.7	Biased loops	No
Diagnostic	3.8	Structured	No
Dhrystone	7.3	Structured	No
crc32	23.0	Highly biased	No

workloads, and a 32× PHT capacity sweep shows the inflation persists at every table size, excluding aliasing.

E. Analytical CPI Model

We derive a first-principles CPI model from the architectural hazard structure of each pipeline variant, following the first-order/mechanistic processor-modeling tradition [30]. Unlike an empirical regression, each term corresponds to a specific physical mechanism:

$$\mathrm{CPI}(D,w) = 1 + \underbrace{\frac{M_w(D)}{N_w}\cdot P(D)}_{\text{branch flush}} + \underbrace{\alpha\cdot L(D)}_{\text{load-use}} + \underbrace{\beta\cdot B(D)\cdot f_b}_{\text{IF2 bubble}} + \gamma_w \tag{1}$$

where D is pipeline depth, w indexes the workload, $M_w(D)/N_w$ is the misprediction rate (mispredictions per instruction; may vary with D due to Mechanism B), $P(D)$ is the flush penalty in cycles (2, 2, 3, 4, 4 for depths 4–8), $L(D)$ is the load-use stall cycles (0, 1, 1, 1, 1.5), $B(D)$ is the IF2 bubble indicator (0, 0, 0, 1, 1), f_b is branch frequency, and γ_w captures workload-specific base overhead from instruction mix, MDU stalls, and data hazards not attributable to pipeline depth. The 8-stage's $L(D) = 1.5$ is a structural average, not a single per-instruction value. Its MEM1/MEM2 split makes a load-use hazard cost 1 cycle when the dependent use is one slot downstream and 2 cycles when adjacent. The effective stall therefore lies between 1 and 2 over the load-use mix. We use the midpoint, and the 8-stage residual ($\leq$0.04 CPI) confirms it suffices.

The general form would add a memory-stall term $\Pi_{\text{mem}} = r_{\text{imiss}}p_{\text{imiss}} + r_{\text{dmiss}}p_{\text{dmiss}}$, which is 0 here (single-cycle memories) but would contribute a depth-independent offset in cache-based systems (Section VI-C).

The shared α (load-use) and β (IF2 bubble) are fitted across all 35 points and γ_w per workload (9 parameters for 35 observations), giving $\alpha = 0.145$, $\beta = 1.12$. Here α is the per-instruction *frequency* of load-use hazards, not a per-stall cost. The value $\alpha < 1$ reflects that only ∼15% of instructions are loads immediately consumed by the next operation, and $\alpha\, L(D)$ multiplies this frequency by the stall cycles each hazard incurs. Three-source forwarding resolves the remaining loads at zero penalty. We note the model's status precisely. Because the branch-flush term takes the *measured* $M_w(D)$ as input, the model *decomposes* observed CPI rather than predicting it ab initio on the two Mechanism-B workloads where M_w varies with D. On the five workloads with depth-invariant M_w it is genuinely predictive from architectural parameters. The LOWO test below isolates the depth-term accuracy from the γ_w offset.

This model achieves $R^2 = 0.977$ across all 35 data points (adjusted $R^2_{\text{adj}} = 0.969$, accounting for the 9 fitted parameters; mean absolute error 0.033 CPI, maximum residual 0.086 CPI), capturing 97.7% of the variance compared to 67% for a simple linear regression.

The model's value is its decomposability: the branch-flush term carries measured (Mechanism A+B) misprediction counts, $\alpha = 0.145$ is the CPI cost per load-use stall cycle, and $\beta = 1.12$ scales the IF2-bubble cost by branch frequency. The per-workload base γ_w ranges from -0.01 (multiply-dominated aha-mont64) to 0.48 (load-heavy edn), capturing instruction-mix effects orthogonal to depth. Because it is orthogonal to depth, γ_w is in principle estimable before implementation from the static instruction profile of the compiled binary (its load, multiply-divide, and branch frequencies) rather than from cycle-accurate simulation. Validating such a static estimator is future work; until then γ_w is fitted. The 4-stage merges MEM/WB, eliminating load-use stalls (it enters the model with $L(D) = 0$) at the cost of a longer critical path. That lengthening is a *frequency* effect (in $F_{\max}$), not a CPI term, and the 4-stage residual stays $\leq$0.04 CPI.

Cross-validation and status. Leave-one-workload-out (LOWO) cross-validation refits on 30 points per fold and predicts the held-out workload from the depth-dependent terms, with γ_w set to the training mean. It gives $R^2_{\text{CV}} = 0.941$ (MAE 0.052 CPI), with α and β stable across folds (0.14 ± 0.01, 1.10 ± 0.05). The largest errors fall on edn and Diagnostic, at the extremes of the instruction-mix distribution. The model is thus *predictive* for its depth-dependent terms (validated out-of-sample) and *descriptive* for the fitted per-workload γ_w; the depth decomposition, not absolute cross-workload CPI prediction, is what we claim. Because α and β barely move across folds, the gap between $R^2_{\text{CV}} = 0.941$ and the in-sample $R^2 = 0.977$ is almost entirely γ_w-substitution error (predicting a held-out workload's base offset from the training mean), not depth-term error. A workload with an extreme instruction mix, such as floating-point-heavy code, would enlarge this γ_w error specifically while leaving the depth terms accurate. Residuals have mean 0 and standard deviation 0.038 CPI with no depth pattern; the largest (+0.086 on 6-stage CoreMark) reflects the smooth $M_w(D)$ slightly under-capturing the depth-6 misprediction jump.

Decomposing CoreMark CPI into the four additive terms of Eq. 1 isolates each architectural cost on top of the ideal 1.00: a depth-invariant base ($\gamma_w = 0.45$); a branch-flush term that grows with depth (0.10 at 4/5-stage to 0.16 at 6-stage and 0.20 at 7/8-stage, Mechanism A, including the Mechanism B count rise at depth 6); a load-use term (0 at 4-stage, 0.15 at 5/6/7-stage, 0.22 at 8-stage); and an IF2-bubble term (0.28, only at 7/8-stage). Summed, these predict measured CPI within the model residual ($\leq$0.09) at every depth across all 35 points.

F. Throughput

Effective throughput (MIPS) is $F_{\max}/\text{CPI}$. This combines two separately obtained quantities: post-place-and-route $F_{\max}$ from static timing analysis and CPI from behavioral simulation. It is therefore a valid figure of merit for architectural comparison across our variants, but not a single end-to-end measurement on running silicon. The two were not co-measured on one physical system, and the product assumes the simulated cycle stream executes at the timing-closed clock. Computed from 10-directive mean $F_{\max}$ and CoreMark CPI, the 6-stage achieves the highest throughput on this fabric (64.1 MIPS on CoreMark, 70.7 MIPS on Diagnostic): its 67% frequency advantage over the 4-stage more than compensates for its higher CPI. The 7-stage reaches 56.9 MIPS (11% below the 6-stage) and the 8-stage 53.9 MIPS, confirming that stages beyond 6 degrade net throughput on this Artix-7 fabric.

V. Speculative GHR Forwarding

Section IV-D established that deeper pipelines degrade branch prediction accuracy through two distinct mechanisms. Mechanism A, higher per-misprediction flush penalty, is inherent to pipeline depth and cannot be reduced without changing the pipeline structure. Mechanism B, inflated misprediction count from stale GHR state, is a deficiency in the predictor's update path, inflating mispredictions by 5.7–10% on branch-dense workloads. This section presents Speculative GHR Forwarding (SGF), a lightweight microarchitectural mechanism that removes the stale-GHR component (Mechanism B) on the workloads that exhibit it.

A. Problem: Stale GHR in Deep Pipelines

The baseline gshare updates its GHR only at branch resolution in EX2, creating a 4-stage update latency on the 7/8-stage pipelines. When branches are separated by fewer than 4 instructions, the second branch uses a GHR that does not reflect the first's outcome.

A cycle-level trace makes the path concrete. Consider two correlated branches B_1, B_2 two instructions apart on the 7-stage pipeline, where B_2's outcome depends on B_1's. B_1 is predicted in IF2 at cycle t and resolves in EX2 at cycle $t{+}4$, writing its outcome into the GHR there. B_2 enters IF2 at cycle $t{+}2$ and is therefore indexed with a GHR that still predates B_1, so $\text{PC}_{B_2}\oplus$GHR selects the PHT counter trained for the *wrong* history context, mispredicting B_2. On the 4/5-stage pipeline the same pair has only a 2-stage update path, B_1 resolves at $t{+}2$ exactly as B_2 is indexed, so B_2 sees fresh history and predicts correctly. The extra misprediction exists purely because depth widened the resolution-to-prediction window. It is a control-path timing artifact, not a capacity or training deficiency. That is why neither a larger PHT nor more training removes it (Section VII-A), while zeroing the update latency does.

Aggregated over the workload, this inflates CoreMark mispredictions by 5.7% at 6-stage and statemate by 10.0% at 7/8-stage relative to the 4/5-stage baseline. These compound with Mechanism A's longer flush penalty to produce a superlinear CPI increase.

B. Design: Dual-GHR Architecture

SGF replaces the single GHR with a dual-GHR architecture:

- **Speculative GHR** (`spec_ghr`): Updated at prediction time (IF2) by shifting in the *predicted* branch direction. This keeps the history fresh for subsequent predictions, even before any branch has resolved.
- **Committed GHR** (`committed_ghr`): Updated at resolution time (EX2) with the *actual* branch outcome. This serves as ground truth for rollback.

On a misprediction, `spec_ghr` is restored from `committed_ghr`, discarding the incorrect speculative history. The speculative GHR is used for PHT indexing at prediction time, while the committed GHR provides the index for PHT updates at resolution time. This separation ensures that the predictor always sees the freshest available history for new predictions while maintaining correct training.

Why not just update the GHR one stage earlier? Moving the update from EX2 to EX1 cuts latency by only one cycle (4 to 3 on the 7/8-stage), which mitigates but does not eliminate Mechanism B. Worse, the update needs the actual outcome from the EX2 branch comparator. Relocating it to EX1 would drag the comparator along and undo the execute-stage split that gives the 56% frequency gain. SGF instead gives the predictor a zero-latency speculative history without relocating any logic.

For precise PHT indexing, a 6-bit GHR checkpoint is forwarded through the pipeline registers alongside each branch instruction, requiring 24 additional flip-flops across the four pipeline stages between IF2 and EX2. On misprediction, the checkpoint stored with the mispredicted branch provides the correct committed GHR state for rollback. The cost scales linearly in the GHR width h and is bounded analytically. Each pipeline stage between IF2 and EX2 carries an h-bit checkpoint ($h{\times}s$ flip-flops for s stages), and rollback is a single h-bit 2:1 multiplexer (`spec_ghr` $\leftarrow$ `committed_ghr`). Both grow linearly, with no quadratic or fan-out-sensitive term. Concretely, a BRAM-backed gshare with a 2048-entry PHT and an 11-bit GHR (the configuration the "free" BRAM predictor implies) needs $11 \times 4 = 44$ checkpoint flip-flops and one 11-bit mux on the 7-stage, versus 24 flip-flops at 6 bits. That is still well under 0.5% of the core. Moving the PHT into a BRAM tile removes its LUTs entirely, so SGF's *relative* overhead shrinks as the predictor grows. Only multi-table predictors, which need a separate checkpoint per history length, grow the cost super-linearly (Section VIII).

C. Integration

SGF requires no changes to the shared RTL modules (ALU, MDU, memory, forwarding, hazard detection). Only the branch predictor module and the pipeline top-level interconnect are modified. The BTB, RAS, and PHT structures are unchanged; only the GHR management logic is replaced. The checkpoint forwarding adds 24 flip-flops (6-bit checkpoint $\times$ 4 pipeline stages). The remaining overhead, the `spec_ghr` and `committed_ghr` registers, the rollback multiplexer, and synthesis-inserted routing registers, accounts for the difference between the 24 architectural FFs and the 57 total additional FFs measured post-place-and-route (Section V-D).

Correctness invariant. SGF maintains the following architectural contract: the PHT is always trained on *actual* branch outcomes, never on speculative ones. Three state elements interact:

- `spec_ghr`: speculative state, updated at prediction time with the predicted direction. Used for PHT *read* indexing only.
- `committed_ghr`: architectural state, updated at resolution time with the actual outcome. Serves as the rollback target.
- `ghr_checkpoint`: a per-branch snapshot of `spec_ghr` at prediction time, forwarded through the pipeline and used for PHT *write* indexing at resolution.

On correct prediction, `spec_ghr` already contains the correct bit and no action is needed. On misprediction, `spec_ghr` is restored from `committed_ghr` (incorporating the actual outcome of the mispredicted branch), and the PHT counter at the *checkpointed* index is updated with the actual direction. This guarantees that (1) the PHT never learns from speculative outcomes, (2) the PHT index used for training exactly matches the index used for the original prediction, and (3) the speculative GHR converges to the committed GHR after every misprediction recovery. The only state that is ever "wrong" is `spec_ghr` between a misprediction and its rollback, a window during which the pipeline is being flushed and no predictions are consumed.

Flush-restore timing. A potential corner case arises when a branch is fetched into IF2 in the cycle immediately following a misprediction flush. In the RTL implementation, the flush signal and the `spec_ghr` restore from `committed_ghr` are both registered at the same clock edge. The flush simultaneously (1) invalidates all pipeline stages between IF1 and EX1, (2) restores `spec_ghr` from `committed_ghr` (shifted by the actual outcome), and (3) redirects the PC to the correct target. Because the flush and restore occur in the same cycle, the next valid instruction to enter IF2 arrives *after* `spec_ghr` has been restored. There is no intermediate cycle in which IF2 could read a partially-updated `spec_ghr`. The speculative GHR shift is also suppressed during the flush cycle via the `!flush` guard, preventing any speculative update from corrupting the just-restored state.

Overlapping branches and the absence of nested rollback. A reorder buffer exists in out-of-order cores (e.g., the Alpha 21264 [7] and MIPS R10000 [8]) precisely to recover speculative state when branches resolve out of program order; SGF needs none because an in-order, single-issue pipeline cannot reach that case. At any instant several branches may be in flight between IF2 and EX2, each carrying its own `ghr_checkpoint` in the pipeline registers. The pipeline issues at most one instruction per cycle and resolves branches strictly in program order at EX2, so only the oldest in-flight branch can resolve, and `committed_ghr` advances solely on that resolution. This admits exactly two transitions for the resolving branch. (i) On a correct prediction, `spec_ghr` already holds the right bit, no rollback occurs, and every younger in-flight checkpoint downstream remains valid. (ii) On a misprediction, the flush squashes *all* younger branches together with their checkpoints in the same cycle, and `spec_ghr` is restored once from `committed_ghr`; no younger speculative state can survive to be "partially" rolled back. Because younger branches are never older than the one resolving, there is no out-of-order analogue in which an earlier branch's misprediction must selectively unwind a later branch's committed update. The single-restore invariant of the previous paragraph therefore covers every reachable state, and forwarded checkpoints are sufficient in all overlapping-branch scenarios, which is the formal reason SGF requires no ROB. These invariants are encoded as a SystemVerilog assertion suite (checkpoint-equals-spec-history, update-uses-checkpoint, no speculative shift during flush, the post-flush restore relation, and no back-to-back flush). The suite is bound into the predictor and checked across all benchmark simulations, and the assertions are part of the released artifact.

D. Synthesis Results

SGF produces no $F_{\max}$ degradation under the same 10-directive methodology (Section III-C): the 7-stage SGF achieves 117.5 ± 2.4 MHz vs. the baseline's 115.0 ± 2.0, and the 8-stage SGF 118.6 ± 1.7 vs. 117.5 ± 1.8. The nominal 2.5 MHz uptick on the 7-stage is within the run-to-run standard deviation ($\sim$2 MHz), and the baseline and SGF 95% confidence intervals overlap. We therefore read it as placement-and-routing noise rather than a real speedup; the defensible claim is that SGF adds no measurable critical-path delay. The area cost is negligible: +49 LUTs / +57 FFs (0.6/0.7%) on the 7-stage and +36 LUTs / +21 FFs (0.5/0.2%) on the 8-stage, with unchanged DSP usage. The speculative-update and checkpoint logic is absorbed by the existing predictor datapath without creating new critical paths.

E. Benchmark Results

Table VI presents the measured misprediction counts and CPI with SGF enabled on the 7- and 8-stage pipelines. All counts are from deterministic behavioral RTL simulation (Vivado XSim), not analytical estimates.

The results show three distinct SGF behaviors:

Strong reduction on branch-dense workloads. CoreMark mispredictions drop from 147,760 to 101,418 on the 7-stage (-31.4%) and from 147,760 to 102,053 on the 8-stage (-30.9%). Statemate drops from 73,263 to 56,627 on both (-22.7%). The aha-mont64 benchmark also shows a 17.2% reduction (139,739 to 115,644) despite exhibiting *no* depth-dependent inflation (Table V classifies it Mechanism-B–free). SGF helps here because the zero-latency speculative GHR is a more accurate history source than the latency-bound committed GHR at *every* depth, not only where Mechanism B is measurable (developed in Section V-F). CPI improvements follow: CoreMark 7-stage drops from 2.02 to 1.96, statemate 7-stage from 1.50 to 1.48.

No effect on sparse-branch workloads. Dhrystone, Diagnostic, and edn show identical misprediction counts with and without SGF. These workloads have branch spacing wide

TABLE VI
SGF MISPREDICTION RESULTS ON 7- AND 8-STAGE PIPELINES (VIVADO XSIM BEHAVIORAL SIMULATION). THESE ARE THE SHIPPED SGF NUMBERS (NO CONFIDENCE FILTER; SEE SECTION V-G). BOLD ENTRIES INDICATE WORKLOADS WHERE SGF REDUCES MISPREDICTIONS BELOW THE 4/5-STAGE BASELINE.

	7-stg Mispred		8-stg Mispred			7-stg CPI		8-stg CPI	
Workload	**Base**	**SGF**	**Base**	**SGF**	**Δ%**	**Base**	**SGF**	**Base**	**SGF**
CoreMark	147,760	**101,418**	147,760	**102,053**	−31/31	2.02	1.96	2.18	2.13
statemate	73,263	**56,627**	73,263	**56,627**	−23/23	1.50	1.48	1.69	1.68
aha-mont64	139,739	115,644	139,739	115,644	−17/17	1.32	1.30	1.32	1.31
crc32	344	511	344	513	+49/49	1.52	1.52	1.56	1.56
edn	10,452	10,452	10,452	10,452	0/0	1.78	1.78	1.95	1.95
Dhrystone	203	203	203	203	0/0	1.85	1.85	1.99	1.99
Diagnostic	69	69	69	69	0/0	1.89	1.89	2.11	2.11

Δ% column shows 7-stage/8-stage misprediction change (rounded). 8-stage baseline counts equal 7-stage (both have 4-stage update latency). **Hardware cost (7-stage, workload-independent):** +49 LUTs (+0.6%), +57 FFs (+0.7%), 0 DSP/BRAM, $F_{\max}$ 117.5 vs. 115.0 MHz (no degradation). Net cycle savings: CoreMark +159,932, statemate +63,222, aha-mont64 +89,044 (others 0).

enough that the committed GHR is already current by the time the next branch is predicted.

crc32 edge case. The crc32 benchmark shows an increase from 344 to 511 mispredictions (+48.5% relative): 167 additional mispredictions out of 174,591 total branches (0.096%), with CPI unchanged at 1.52. Section V-G dissects this interaction and the confidence-filter countermeasure we measured but deliberately rejected (it would fix crc32 but costs more CoreMark benefit than it saves).

F. Key Finding: Deep-Pipeline Accuracy Below Shallow-Pipeline Baseline

SGF does not merely restore deep-pipeline accuracy to the shallow-pipeline level. It pushes misprediction counts *below* the 4/5-stage baseline:

- CoreMark: 101,418 (7-stg SGF) vs. 145,735 (4/5-stg baseline), 30.4% *fewer* mispredictions than the shallowest pipeline.
- statemate: 56,627 vs. 66,603, 15.0% fewer.
- aha-mont64: 115,644 vs. 139,739, 17.2% fewer.

This result follows directly from the update-latency asymmetry: the speculative GHR has zero update latency (it incorporates the predicted direction in the same cycle as the prediction), while even the shallowest baseline has 2-stage update latency. When predictions are mostly correct (79.8% on CoreMark), the speculative GHR provides a more accurate approximation of true branch history than any committed GHR operating with nonzero latency. This means SGF is beneficial at *any* pipeline depth where the predictor uses global history, not only at depths where Mechanism B is measurable.

This also resolves an apparent paradox. The 31.4% reduction far exceeds the +5.7% Mechanism B inflation measured at the 6-stage because that figure captures only the *marginal* latency added by deepening (2- to 3–4-stage update path). SGF instead removes the *entire* update latency, including the 2-stage component shared by every baseline depth. Mechanism B is thus the depth-dependent slice of a larger latency cost; SGF's benefit is bounded by total update latency, not the smaller depth-marginal inflation the controlled experiments isolate.

CPI impact. The 31% misprediction reduction yields a 3% CPI improvement (2.02→1.96 on the 7-stage). Mechanism B is only one of four CPI components, alongside Mechanism A (∼0.20 CPI), the IF2 bubble (∼0.28), and load-use stalls (∼0.15). SGF eliminates Mechanism B fully; the remaining components are inherent to the pipeline structure and lie outside SGF's scope.

G. crc32 Edge Case and the Confidence-Filter Tradeoff

One workload, crc32, shows a misprediction *increase* under SGF. We treat it as a bounded, fully-explained edge case rather than a counterexample. The increase is 167 mispredictions out of 174,591 branches (0.096%) and changes CPI by less than 0.01 (1.52 in both cases). Its cause is a single identifiable interaction described below. No workload, crc32 included, shows a CPI regression. We dissect it because predicting exactly where SGF is and is not neutral is part of characterizing the mechanism. Stated as a class: SGF can *raise* the raw misprediction count on workloads dominated by saturated-counter, highly-biased branches, where the baseline's update latency was accidentally beneficial. This is a *structural* edge case, predictable from the PHT-saturation interaction, not a rare anomaly; across our suite it remains CPI-neutral.

Structural mechanics. The crc32 inner loop has one dominant branch (`if (crc & 1)`), taken 99.8% of iterations, so its PHT counter saturates and the committed GHR holds a long run of `1`s. Under the baseline, a rare not-taken misprediction does not reach the GHR until EX2, after the next iteration has already been predicted with clean (stale but accidentally accurate) history, so update latency acts as an accidental filter. SGF instead shifts the wrong `0` into `spec_ghr` immediately, so the next prediction uses a polluted PC XOR GHR index (a different, possibly untrained counter). This adds 1–3 mispredictions until correct predictions or the EX2 rollback restore the pattern, 167 in total over 174,591 branches (0.096%).

A confidence filter would close it, at a cost. Suppressing speculative updates when the PHT counter is saturated (one 2-bit comparator, no flip-flops) restores crc32 to its baseline 344 mispredictions. It also forfeits useful freshening on stably-biased hot branches, however: on CoreMark it raises

mispredictions from 101,418 to 117,207, cutting SGF's reduction from 31.4% to 20.7%. The trade is lopsided: a 167-misprediction, zero-CPI blip on crc32 against ∼16,000 real CoreMark savings. We therefore ship SGF without the filter (crc32 CPI unchanged at 1.52, the extra flushes negligible against the loop runtime) and retain it only as a build-time option for crc-like, highly-biased workloads.

Operating regime. SGF benefits workloads whose branch outcomes vary across short windows (CoreMark, statemate, aha-mont64), is neutral when branches are widely spaced (edn, Diagnostic) or already accurate (Dhrystone), and is CPI-neutral on highly biased loops (crc32) despite a small misprediction blip. No workload shows CPI degradation.

H. SGF Cost-Benefit Summary

Across all seven workloads on the 7-stage pipeline (Table VI), SGF produces a net benefit on three (CoreMark, statemate, aha-mont64), is neutral on three (Dhrystone, Diagnostic, edn), and has a marginal, CPI-neutral negative effect on one (crc32). The hardware cost is workload-independent and small (+49 LUTs / +57 FFs, 0.6–0.7%, no $F_{\max}$ loss); the largest single benefit is CoreMark's 159,932-cycle saving.

I. SGF at the 6-Stage Design Point

Sections V-D–V-H evaluated SGF on the 7- and 8-stage pipelines, where Mechanism B is largest. Because the 6-stage is the highest-throughput depth on this fabric (Section IV-F), we also implemented SGF on the 6-stage variant. It is the same dual-GHR mechanism, with the speculative-history checkpoint forwarded through the shorter IF/ID/EX1/EX2 path, evaluated on the two Mechanism-B workloads. On CoreMark, SGF reduces 6-stage mispredictions from 154,077 to 117,207 (−23.9%, CPI 1.83→1.79). On statemate, it reduces them from 69,933 to 54,329 (−22.3%, CPI 1.37→1.35). Instruction and branch counts are identical to the baseline, confirming the implementation is performance-only.

The reduction is substantial at the recommended design point, though smaller than at 7/8-stage (−31.4% on 7-stage CoreMark). The 6-stage's 3-stage update latency leaves less stale-GHR inflation to correct than the 4-stage latency of the deeper pipelines, consistent with Mechanism B scaling with depth. On the workloads that exhibit no Mechanism B (aha-mont64, crc32, edn), the 6-stage SGF shows small, CPI-neutral misprediction increases on highly biased branches, the same edge case discussed in Section V-G. SGF is therefore beneficial at every depth with nonzero update latency, and most where Mechanism B is largest.

J. Compiler-Optimization Sensitivity (`-O2`)

A key robustness question is whether aggressive optimization, by unrolling and inlining, widens inter-branch distance enough to dissolve Mechanism B. We rebuilt CoreMark and statemate at `-O2` and re-ran the 7-stage baseline and SGF. `-O2` reshapes the workloads (CoreMark branches −25.8%, IBD 4.0→4.5; statemate IBD 8.4→7.5) but both stay under the IBD ≈ 10 threshold, so Mechanism B persists and SGF still helps: CoreMark 141,212 → 79,705 (−43.6%, CPI 1.86→1.76) and statemate 83,257 → 73,267 (−12.0%, 1.44→1.43). The CoreMark reduction is in fact *larger* than at `-O1` (−31.4%), so optimization does not remove SGF's use case for these branch-dense kernels. statemate moves the other way: `-O2` both raises its baseline mispredictions (73,263 → 83,257) and roughly halves SGF's relative benefit (−22.7% to −12.0%). SGF's gain is therefore optimization-sensitive in magnitude, though its sign and CPI-neutrality hold under both settings.

K. Generalization to Other Correlation Predictors

SGF is developed on gshare, but its mechanism is predictor-agnostic. It applies wherever a prediction reads a history register that the branch updates only after some latency. The fix is always the same: a speculative copy, updated at prediction, forwarded as a per-branch checkpoint, and restored on misprediction. Two properties carry across topologies unchanged. The in-order single-issue pipeline admits no nested rollback (Section V-C), so one checkpoint per in-flight branch suffices and no ROB is needed. The cost stays linear in the speculative-history width carried per branch: that width across the IF2-to-EX2 span, plus one same-width rollback multiplexer.

To test this beyond gshare, we implemented and measured two further predictor families on the 7-stage pipeline, each in baseline and SGF variants. The first is a tournament predictor: a global gshare PHT and a PC-indexed local PHT, arbitrated by a GHR-indexed chooser. The second is a downscaled TAGE predictor [20]: a bimodal base plus three tagged components with geometric global-history lengths of 4, 8, and 16 bits. Table VII reports the misprediction change for each. Instruction and branch counts are bit-identical between every baseline and its SGF variant (the predictor is performance-only), so the deltas are exact. Mechanism B reproduces on both predictors, and SGF reduces mispredictions on the history-dense workloads in every case.

The magnitude tracks how history-dependent each predictor is. The tournament reductions are smaller than standalone gshare's because the chooser routes a share of branches to the staleness-immune local component. statemate is the clearest case. Stale history had pinned the tournament's global component at bimodal accuracy (63,274 vs. the 63,272 bimodal control of Section VII-A2). SGF lifts it to 56,619, within eight mispredictions of standalone gshare+SGF (56,627). TAGE, whose three tagged tables are all history-indexed, has the most stale-history headroom and matches gshare's CoreMark reduction (−31.1%).

The biased-branch edge case of Section V-G also generalizes, but it relocates with predictor structure rather than staying on one workload. On gshare it raises crc32 mispredictions. On TAGE it instead raises aha-mont64 (+5.0%, CPI-neutral), whose biased loops let the speculative shift pollute the history-indexed tables, while crc32 itself improves. The saturated-provider confidence filter that removes the gshare crc32 increase (Section V-G) does *not* transfer to TAGE: applied here, we measured that it *worsens* the aha-mont64 case (to +12.2%) and erodes the CoreMark reduction (to

TABLE VII
SGF MISPREDICTION CHANGE (Δ) ACROSS THREE PREDICTOR FAMILIES ON THE 7-STAGE PIPELINE (ARTIX-7). INSTRUCTION AND BRANCH COUNTS ARE IDENTICAL BETWEEN EACH BASELINE AND ITS SGF VARIANT, SO THE DELTAS ARE EXACT. SECTION V-K INTERPRETS THE TRENDS.

Workload	gshare	Tournament	TAGE
CoreMark	-31.4%	-3.1%	-31.1%
statemate	-22.7%	-10.5%	-11.1%
aha-mont64	-17.2%	-2.8%	$+5.0\%$
crc32	$+48.5\%$	-48.3%	-48.8%

-20.1%), because holding the speculative shift across TAGE's three history-indexed tables starves them of fresh history. We therefore report the unfiltered TAGE counts and treat the small, CPI-neutral aha-mont64 increase as an accepted limitation on biased-loop workloads.

The hardware cost, in contrast, does not generalize uniformly, and the measurements confirm the multi-table caveat of Section V-B. On the tournament predictor SGF stays cheap: $+50$ LUTs (0.6%), $+44$ flip-flops (0.5%), and no frequency penalty (112.6 vs. 111.7 MHz across three seeds), since one forwarded global checkpoint corrects both the global PHT and the chooser. On TAGE the checkpoint flip-flops stay modest ($+74$), but LUTs grow $+2{,}434$ ($+21\%$). Each of the three geometric-history tables hashes its index and tag from the speculative history for the read and from the forwarded checkpoint for the write. That per-table dual hashing does not share. This is the super-linear growth anticipated in Section V-B, now measured. TAGE is also expensive before SGF (about $1.5\times$ the gshare core's LUTs and a frequency tier slower, $\sim$94 MHz), so it is not the predictor a resource-constrained soft core would choose on this fabric. SGF delivers a misprediction reduction at negligible cost on the single-global-history predictors (gshare, tournament) that suit such cores; on a multi-table predictor the benefit persists but the low cost does not.

VI. DISCUSSION

A. Pipeline Partitioning Boundaries on Artix-7

The data identify one partitioning boundary that matters on this fabric: splitting the monolithic execute stage into EX1 (forwarding) and EX2 (computation). This split produces a 56% frequency improvement (74 to 117 MHz). No other split, IF into IF1/IF2, MEM into MEM1/MEM2, produces a statistically significant frequency gain (within-tier differences are $<2\%$). The practical design question on Artix-7 is therefore binary: split the execute stage (gain 56% frequency at $\sim$0.12 CPI per added stage) or leave frequency capped regardless of depth. This contrasts with ASICs, where $F_{\max}$ rises roughly monotonically with depth because gate delay scales with logic levels [1]. On FPGAs, function-independent LUT delay and undiminished routing delay [5] mean only breaks of LUT-dominated chains (the execute stage) help. The boundary is thus specific to Artix-7's 6-input LUT fabric and Vivado 2025.2; a different LUT depth, routing topology, or process node could move it. The CPI model's per-stage costs, however, are hazard-driven and fabric-independent, so the model and SGF's misprediction reduction transfer to other FPGA families; only the frequency tier boundaries shift.

B. Physical Critical Path Analysis

The two-tier structure has a concrete physical cause. In the 4/5-stage designs the execute stage chains the forwarding mux, the ALU (32-bit CARRY4 adder $\sim$3.2 ns plus barrel shifter), and the branch comparator on one path ($\sim$8.5 ns: $\sim$4.1 ns logic, $\sim$4.4 ns routing, i.e. $\sim$74 MHz). The 6-stage EX1/EX2 split registers mid-path, halving per-stage logic delay and dropping the path to $\sim$3.5 ns ($\sim$117 MHz). Subsequent IF/MEM splits yield no gain because those paths are routing-dominated ($\sim$1.5 ns LUT vs. $\sim$4.0 ns routing), and register insertion does not reduce routing delay. This asymmetry is what produces the two tiers.

C. Cache Miss Sensitivity Analysis

All benchmarks execute from single-cycle memory (near-100% hit). In production, cache misses add a depth-independent term $\mathrm{CPI}_{\mathrm{total}} = \mathrm{CPI}_{\mathrm{core}} + r_{\mathrm{miss}}\, p_{\mathrm{miss}}$ (miss rate $\times$ penalty) that dilutes branch-prediction gains. SGF's *absolute* cycle saving on 7-stage CoreMark (159,932 flush cycles) is miss-rate-independent, so its *relative* benefit shrinks as memory stalls grow: 3.0% at zero misses, to 2.7% (1%/20-cycle), 2.0% (2%/50-cycle), and 0.9% (5%/100-cycle). SGF is thus most valuable in embedded systems with tightly coupled memories, the primary FPGA-soft-core use case, and least where cache-miss penalties dominate.

D. Anticipated Objections

Three reading caveats, separating what generalizes from what does not. *(i) Architectural fact vs. workload-specific result vs. heuristic.* The *architectural fact* is the mechanism: Mechanism B arises for any global-history predictor with nonzero update latency, shown causally by the bimodal and PHT controls, so the decomposition and the ROB-free construction transfer. The *magnitudes* are workload- and predictor-specific: the percentages reported here hold for this benchmark set and predictor, not universally. The IBD threshold is a *heuristic*, not a validated rule, grounded in only two positive workloads (Section IV-D). *(ii) Significance and scope:* SGF is a cost-benefit improvement, not a throughput gain (Section I), and the optimal-depth conclusion holds under the stated `-O1`, benchmark, and single-cycle-memory boundaries (Sections VI-C, VIII), not beyond them.

E. Design Guidance for Practitioners

For RV32IM-class soft cores on Artix-7, the data support four guidelines. The 6-stage is the highest-throughput depth here (64.1 MIPS CoreMark): it captures the full execute-split frequency benefit at moderate CPI. The 5-stage is dominated, sharing the 4-stage frequency ceiling while adding load-use stalls, so it helps only when area is binding (a conclusion conditional on the `-O1` mix). The 7/8-stage match the 6-stage on frequency but pay higher CPI, justified only when the IF

split resolves a genuine timing bottleneck such as synchronous BRAM instruction memory. SGF does *not* let a deeper pipeline out-throughput the 6-stage (7-stage+SGF: 58.7 MIPS vs. 64.1). It is worth adding at any forced depth with $\geq$3 stages of update latency, including the 6-stage ($-$23.9% CoreMark), for 0.5–0.7% area against a 17–31% misprediction cut.

VII. Causal Isolation and Supporting Analyses

A. Causal Isolation of Mechanism B

The baseline measurements in Section IV-D established a correlation between pipeline depth and misprediction count on branch-dense workloads. Two alternative explanations must be excluded before attributing this to GHR staleness: (1) the inflation could be an aliasing artifact of the 64-entry PHT, resolvable by increasing table capacity; or (2) some other pipeline-structural effect (e.g., forwarding interactions) could cause additional mispredictions at deeper depths independent of the predictor. We design two controlled experiments to rule out each alternative.

1) PHT Capacity Sweep on CoreMark and statemate: We swept the PHT across six sizes from 32 to 1024 entries on all five pipeline depths for the two workloads that exhibit Mechanism B, CoreMark and statemate (60 simulation runs). Table VIII presents four representative sizes (the 32$\times$ extremes plus the 64-entry default and 256-entry midpoint); the trends hold at the omitted 128- and 512-entry points.

statemate (controlled result). We treat statemate as the load-bearing result of this experiment. Its branch outcomes are data-dependent, yet its branch pattern is sparse enough (IBD = 8.4) that aliasing does not dominate, so the depth effect is observed against a stable baseline. Across a 32$\times$ capacity range (PHT=32 to 1024), the misprediction inflation at 7/8-stage depth is +10.0% at every PHT size from 32 to 256. Absolute counts vary by fewer than 8 mispredictions out of 73,000. At PHT=1024 the larger table resolves some aliasing and lowers the absolute baseline. Yet the depth-dependent inflation does not merely persist; it grows to +11.8%. No PHT capacity from 32 to 1024 entries closes the depth gap. This is the expected signature of an effect that is not caused by table capacity: a 32$\times$ sweep cannot remove what aliasing did not create.

CoreMark (high-aliasing corroboration). CoreMark has the densest branch pattern of any workload (IBD = 4.0) and therefore the highest aliasing pressure. Its absolute baseline counts swing non-monotonically with table size (145K–172K; the 64-entry table happens to alias favorably, the 256-entry table unfavorably). This aliasing noise partially masks the depth effect, so we deliberately do *not* rest the causal claim on CoreMark's inflation magnitude. Two facts nonetheless survive the aliasing: the 6-stage exceeds the 4/5-stage baseline at *every* one of the six table sizes, and the 7/8-stage shows a consistent +0.7–1.9% inflation at every size. The contrast between the two workloads is itself diagnostic. The table-size-dependent baseline swing isolates the *aliasing* axis, while the table-size-invariant depth gap isolates the *staleness* axis; the two are orthogonal. A larger table reduces absolute mispredictions but does not correct the stale-GHR inflation that SGF targets, and only the latter is the subject of this paper.

2) Bimodal Predictor Control Experiment: To confirm that Mechanism B requires a GHR, we replaced the gshare predictor with a bimodal predictor that indexes the PHT using only the PC (no global history register). We ran CoreMark and statemate on all five pipeline depths. The bimodal predictor produces *exactly* the same misprediction count at every depth on both workloads: 85,493 (CoreMark) and 63,272 (statemate), not a single additional misprediction at any depth. On the same workloads and pipelines, gshare shows +5.7% inflation on CoreMark at 6-stage and +10.0% on statemate at 7/8-stage (Table VIII). The only architectural difference is the presence of a GHR in the prediction index.

Causal conclusion. Together these two controlled experiments isolate the cause:

1) The bimodal control shows that Mechanism B *requires* a GHR: without one, no depth-dependent misprediction variation occurs on any workload.
2) The PHT capacity sweep shows that Mechanism B is *not* an aliasing artifact: the inflation persists at every table size from 32 to 1024 entries.
3) These two controls rule out the alternative explanations, leaving stale GHR update latency as the cause; SGF removes that latency and is the matching fix.

Is SGF better than abandoning global history? For area-constrained designs the bimodal counts answer whether a correlation predictor with SGF beats a GHR-free bimodal of the same size, and the answer is workload-dependent. On statemate global history pays off: gshare+SGF (56,627) $<$ bimodal (63,272) $<$ stale-GHR gshare (73,263), so SGF makes the predictor worth its GHR. On CoreMark the 64-entry gshare is aliasing-bound and bimodal (85,493) beats gshare even with SGF (101,418), since SGF removes staleness but not table-capacity aliasing. The guidance is thus not universal: SGF is worth it when global history helps; where a small table leaves gshare aliasing-bound, a bimodal predictor can be the better choice.

B. BRAM Instruction Memory Effect

To test whether the 7-stage IF split becomes beneficial with synchronous memory, we synthesized BRAM variants of all five pipelines using a registered-output instruction memory that infers Block RAM. With BRAM the deep tier rises to 121.5 MHz (6-stage), 120.9 MHz (7-stage), and 121.2 MHz (8-stage). The 7/8-stage improve from their LUT-memory 115.0/117.5 MHz to nearly match the 6-stage, while the shallow tier holds (4-stage 69.3, 5-stage 73.7 MHz). The 4-stage dips slightly because the registered BRAM output lengthens its already-critical execute path. The IF1/IF2 boundary naturally absorbs the BRAM's 1-cycle read latency, making the 7-stage frequency-competitive; but its deeper-pipeline CPI overhead remains, so it still needs SGF and moderate-branch-density workloads to match the 6-stage's throughput. This also answers the memory-mapping concern: the main study uses distributed LUT RAM to hold memory organization *invariant* across depths, and these BRAM variants are the measured counterfactual with true synchronous-read primitives. Mapping to BRAM shifts frequency but not the CPI ordering, so

TABLE VIII
MISPREDICTION COUNTS (RTL SIMULATION) ACROSS PHT SIZES AND PIPELINE DEPTHS ON COREMARK AND STATEMATE (GSHARE PREDICTOR). Δ SHOWS INFLATION RELATIVE TO THE 4/5-STAGE BASELINE AT EACH PHT SIZE.

	PHT=32		PHT=64		PHT=256		PHT=1024	
Depth	**CM**	**SM**	**CM**	**SM**	**CM**	**SM**	**CM**	**SM**
4-stg	150,502	66,599	145,735	66,603	172,161	66,607	164,997	56,623
5-stg	150,502	66,599	145,735	66,603	172,161	66,607	164,997	56,623
6-stg	177,857	69,929	154,077	69,933	179,277	69,935	166,945	63,282
7-stg	152,376	73,259	147,760	73,263	174,436	73,266	166,934	63,283
8-stg	152,376	73,259	147,760	73,263	174,436	73,266	166,934	63,283
Δ 6-stg	+18.2%	+5.0%	+5.7%	+5.0%	+4.1%	+5.0%	+1.2%	+11.8%
Δ 7/8-stg	+1.2%	+10.0%	+1.4%	+10.0%	+1.3%	+10.0%	+1.2%	+11.8%

the 6-stage stays the throughput optimum and a real-BRAM mapping strengthens rather than invalidates it.

C. Power and Efficiency

Power is estimated with Vivado's `report_power` on each out-of-context–synthesized variant. The estimate is annotated with workload-specific switching activity, captured as a Switching Activity Interchange Format (SAIF) file from a behavioral CoreMark simulation of that variant. These are Vivado vector-based estimates at "Medium" confidence, not physical measurements, and can differ from silicon by roughly ±20–30%. We therefore rely on the *relative ordering across depths*, not the absolute watts, and report no hardware-power validation. Under workload-specific activity, total on-chip power *falls* with depth, from 0.235 W (4-stage) to 0.217 W (8-stage); static power is constant at 0.069 W and dynamic power drops from 0.167 W to 0.149 W. This trend is opposite to a uniform-toggle approximation, which only a workload-derived SAIF captures. Although deeper pipelines add registers, their lower IPC (more bubbles and stalls per cycle) reduces average switching activity, so on the same workload they draw *less* dynamic power.

From energy $E = P_{\text{dyn}} \cdot \text{Cycles}/F_{\max}$ and EDP $= E \cdot \text{Cycles}/F_{\max}$, the 6-stage achieves the lowest energy per CoreMark run (7.0 mJ) and the lowest EDP (318 mJ·ms), versus 10.6 mJ/675 for the 4-stage and 8.0 mJ/432 for the 8-stage. It also has the highest efficiency: 286 MIPS/W, versus 253/248 for the 7/8-stage and 195/197 for the 4/5-stage. The 6-stage is thus the most energy-efficient depth as well as the highest-throughput, though we present EDP as a relative comparison, not a measured criterion. SGF's +49 LUT/+57 FF add $<1\%$ to dynamic power.

D. Published Cores Context

As a sanity check, our 70–118 MHz range sits in a realistic mid-range among the Artix-7 RISC-V soft cores of Table I. We make no cross-project performance claim: those cores differ in ISA subset, execution model, predictor, and memory, so their frequency gaps reflect those confounders, not pipeline depth.

VIII. LIMITATIONS

The boundaries of this study constrain every conclusion drawn above and must be stated precisely.

Fabric and tool dependence. All synthesis, timing, and power data are from Artix-7 XC7A35TCPG236-1 with Vivado 2025.2. The two-tier $F_{\max}$ structure is a property of the 6-input LUT fabric and Vivado's placer/router. A different LUT architecture (e.g., Intel ALM), routing topology, process node, or tool (Quartus, Yosys+nextpnr) could shift the execute-split knee to a different depth or eliminate it. The absolute $F_{\max}$/area/power figures do not transfer without re-synthesis. The 10-directive methodology mitigates single-run bias within Vivado but does not address cross-tool variation.

Single microarchitecture and memory scope. The core is a research vehicle lacking the C extension, out-of-order execution, multi-level caches, TLB/MMU, and I/O. A different forwarding topology, tournament predictor, superscalar issue, or deeper memory hierarchy could yield a different optimal depth and SGF impact. All workloads fit in 16 KB of instruction memory at near-100% hit rate. The cache-miss bound (Section VI-C) is analytical, not measured against a real hierarchy, so systems with external DDR or multi-level caches need re-evaluation.

CPI model. The model (Eq. 1) predicts the depth-dependent terms from architectural parameters, but its per-workload base γ_w is fitted empirically and remains descriptive, not first-principles. The model is validated only on the seven workloads used for fitting.

Predictor scope. SGF is measured on three predictor families: single-table gshare, a global/local tournament predictor, and a downscaled TAGE predictor (Section V-K). The stale-GHR effect is not gshare-specific, as any global-history-indexed predictor incurs it. On TAGE [20] the misprediction benefit persists but the area cost grows to $+21\%$ LUTs, confirming the multi-table caveat. A measured evaluation on perceptron [21] and local-history [6] predictors is future work. The crc32 confidence filter (Section V-G) ships off by default.

Benchmark and compiler scope. The seven workloads span synthetic, industry-standard, and embedded categories (branch frequency 4.4–26.3%, 1.4K–4.5M instructions) but exclude floating-point, OS-level, and multi-threaded code. The IBD indicator (Table V) is a preliminary correlate of Mechanism B susceptibility, derived from only two positive workloads and not validated out-of-sample. Headline results use `-O1`, but we re-measured CoreMark and statemate at `-O2` (Section V-J): both stay under the IBD threshold, and

Mechanism B and SGF's benefit persist (CoreMark's reduction in fact grows), so the effect is not an `-O1` artifact. `-O3`, floating-point, OS-level, and multi-threaded workloads remain untested, and the IBD indicator is still grounded in only two positive workloads.

IX. Conclusion

This paper makes two contributions to pipelined FPGA soft-core design. First, a controlled depth experiment (five RV32IM variants from shared RTL on Artix-7) separates depth-dependent CPI into an inherent flush penalty (Mechanism A) and a correctable stale-GHR penalty (Mechanism B, up to 10%). Two controls isolate the latter causally: a no-GHR bimodal predictor shows zero inflation, and a 32× PHT sweep shows inflation at every size. A first-principles CPI model captures both ($R^2 = 0.977$, $R^2_{\text{CV}} = 0.941$). It also identifies a two-tier $F_{\max}$ structure ($p < 0.001$, $d = 18.5$) whose sole knee is the execute-stage split, making the 6-stage the highest-throughput depth. We present this as fabric- and tool-conditional, not universal.

Second, Speculative GHR Forwarding eliminates Mechanism B with a ROB-free dual-GHR architecture that updates speculatively at prediction time and rolls back from pipeline-register checkpoints. It reduces 7-stage CoreMark mispredictions by 31.4% and statemate by 22.7% at 0.5–0.7% area and no frequency penalty, pushing deep-pipeline accuracy below the shallow baseline. No workload shows CPI degradation; the crc32 blip is CPI-neutral. For practitioners, SGF is a low-risk addition to any gshare-class in-order pipeline with three or more stages of update latency.

What transfers without re-measurement is the Mechanism-A/B decomposition and the ROB-free construction, not the specific percentages. Measured tournament and TAGE predictors (Section V-K) confirm the benefit reproduces across predictor families, at sub-1% cost on single-history predictors and a larger cost on multi-table TAGE. An `-O2` re-measurement (Section V-J) confirms Mechanism B and SGF's benefit survive aggressive optimization. Future work extends this to `-O3` and floating-point/OS workloads, to perceptron and local-history predictors, and to other FPGA fabrics and physical-hardware power/cycle validation. The complete RTL, testbenches, synthesis scripts, and benchmark programs are available at https://github.com/devanshjoshi08/RISCV-Pipeline-Research.

References

[1] M. S. Hrishikesh, N. P. Jouppi, K. I. Farkas, D. Burger, S. W. Keckler, and P. Shivakumar, "The optimal logic depth per pipeline stage is 6 to 8 FO4 inverter delays," in *Proceedings of the 29th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2002, pp. 14–24.

[2] E. Sprangle and D. Carmean, "Increasing processor performance by implementing deeper pipelines," in *Proceedings of the 29th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2002, pp. 25–34.

[3] A. Hartstein and T. R. Puzak, "The optimum pipeline depth for a microprocessor," in *Proceedings of the 29th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2002, pp. 7–13.

[4] Xilinx Inc., *7 Series FPGAs Configurable Logic Block User Guide (UG474)*, Xilinx Inc., San Jose, CA, 2018, v1.8.

[5] I. Kuon and J. Rose, "Measuring the gap between FPGAs and ASICs," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 26, no. 2, pp. 203–215, 2007.

[6] T.-Y. Yeh and Y. N. Patt, "Two-level adaptive training branch prediction," in *Proceedings of the 24th Annual International Symposium on Microarchitecture (MICRO)*. ACM/IEEE, 1991, pp. 51–61.

[7] R. E. Kessler, "The Alpha 21264 microprocessor," *IEEE Micro*, vol. 19, no. 2, pp. 24–36, 1999.

[8] K. C. Yeager, "The MIPS R10000 superscalar microprocessor," *IEEE Micro*, vol. 16, no. 2, pp. 28–40, 1996.

[9] R. Gonzalez and M. Horowitz, "A power-aware methodology for designing processor pipelines," *IEEE Journal of Solid-State Circuits*, vol. 34, no. 3, pp. 345–354, 1999.

[10] A. Waterman and K. Asanović, *The RISC-V Reader: An Open Architecture Atlas*. Berkeley, CA: Strawberry Canyon LLC, 2019.

[11] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface, RISC-V Edition*, 2nd ed. Cambridge, MA: Morgan Kaufmann, 2020.

[12] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA: Morgan Kaufmann, 2017.

[13] O. Kindgren, "SERV: The SErial RISC-V CPU," https://github.com/olofk/serv, 2019, accessed May 2026.

[14] C. Wolf, "PicoRV32: A size-optimized RISC-V CPU," https://github.com/YosysHQ/picorv32, 2017, accessed May 2026.

[15] P. D. Schiavone, D. Rossi, M. Gautschi, A. Pullini, F. Conti, and L. Benini, "Slow and steady wins the race? A comparison of ultra-low-power RISC-V cores for Internet-of-Things applications," in *27th International Symposium on Power and Timing Modeling, Optimization and Simulation (PATMOS)*. IEEE, 2017, pp. 1–8.

[16] C. Papon, "VexRiscv: A RISC-V implementation written in SpinalHDL," https://github.com/SpinalHDL/VexRiscv, 2018, accessed May 2026.

[17] Western Digital Corporation, "SweRV EH1 RISC-V core," https://github.com/chipsalliance/Cores-SweRV, 2019, accessed May 2026.

[18] F. Zaruba and L. Benini, "The cost of application-class processing: Energy and performance analysis of a Linux-ready 1.7-GHz 64-bit RISC-V core in 22-nm FDSOI technology," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 27, no. 11, pp. 2629–2640, 2019.

[19] S. McFarling, "Combining branch predictors," Digital Western Research Laboratory, Tech. Rep. WRL Technical Note TN-36, 1993.

[20] A. Seznec, "A new case for the TAGE branch predictor," in *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. ACM, 2011, pp. 117–127.

[21] D. A. Jiménez and C. Lin, "Dynamic branch prediction with perceptrons," in *Proceedings of the 7th International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2001, pp. 197–206.

[22] A. Seznec, "Revisiting the perceptron predictor," IRISA, Tech. Rep. PI-1878, 2006.

[23] P. Michaud, A. Seznec, and R. Uhlig, "Trading conflict and capacity aliasing in conditional branch predictors," in *Proceedings of the 26th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 1999, pp. 292–303.

[24] M. Biglari, S. M. Fakhraie, and M. T. Manzuri, "A survey on pipelining techniques in RISC processors," in *IEEE 10th International Conference on Computing, Communication and Networking Technologies (ICCCNT)*. IEEE, 2019, pp. 1–6.

[25] K. Asanović, R. Avižienis, J. Bachrach, S. Beamer, D. Biancolin, C. Celio, H. Cook, D. Dabbelt, J. Hauser, A. Izraelevitz *et al.*, "The rocket chip generator," EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2016-17, 2016.

[26] J. E. Smith, "A study of branch prediction strategies," in *Proceedings of the 8th Annual Symposium on Computer Architecture (ISCA)*, 1981, pp. 135–148.

[27] D. A. Patterson, "Reduced instruction set computers," *Communications of the ACM*, vol. 28, no. 1, pp. 8–21, 1985.

[28] Embench Task Group, "Embench: A modern embedded benchmark suite," https://github.com/embench/embench-iot, 2019, accessed May 2026.

[29] EEMBC, "CoreMark: An EEMBC benchmark," https://www.eembc.org/coremark/, 2009, accessed May 2026.

[30] T. S. Karkhanis and J. E. Smith, "A first-order superscalar processor model," in *Proceedings of the 31st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2004, pp. 338–349.